DS34 Competition 1 Report

組員:施淳譯 蔡譯穎 陳亭蓁

A、資料分析以及前處理

X_train

	A	B	C
1	Feature_1	Feature_2	Feature_3
2	-2.54E-06	1	0
3	0.38532	1	1
4	-0.08365	1	0
5	-0.38532	1	0
6	-2.54E-06	1	0
7	0.524397	1	0
8	-0.72791	1	0
9	-0.72791	1	0
10	-0.5244	1	0

y_train

	A
1	target
2	1
3	0
4	1
5	0
6	1
7	1
8	1
9	1
10	1

在Competition 1 中，要進行的是二分類任務，並使用roc_auc score來評定最後的預測結果。我們所拿到的X_train中會有數值型以及類別型的特徵，數值型特徵中的數值是有包含小數點(float/double)的，如上圖X_train中的Feature_1，類別型特徵則是整數的資料(int)，如上圖X_train中的Feature_2、Feature_3，在y_train中則會有對應的類別(0/1)，如上圖y_train中的target欄位。在X_test中我們會拿到與X_train相同的特徵數量，用來預測y_test，並以y_test的roc_auc score進行評分。

觀察資料，我們發現了以下幾件事:

1.每個資料集中包含的特徵結構不一定相同，可能有不同的特徵數量、特徵的類別(數值型/類別型)、特徵的範圍

2.每個數值型特徵的數值範圍不一定相同

3.每個類別型特徵的數值範圍不一定相同，兩個類別型特徵的Feature下可能有不同的類別數量(ex.Feature_2的類別有0~3，Feature_3有2~6)

4.y_train中的類別可能是不平衡的，target中0/1的數量可能不相同

5.每個dataset中的X_train資料筆數不相同，部分資料集會有很少的訓練資料

為了提升我們的預測成果，我們可以先對資料進行特徵工程，來幫助模型達到更好的訓練及預測結果，我們採用的特徵工程如下:

1.對於數值型特徵範圍不相同的問題，在部分模型(ex.SVM、MLP或是Fully Connected Neural Network)可能會對過大或是過低的數值特別敏感，因此我們先採用了Standard Scalar或是MinMaxScalar來調整數值型特徵的範圍

2.對於類別型的特徵我們也進行了類似的調整，但因為在我們的資料集中類別型特徵是使用數值代表類別，我們不能採取與數值型特徵相同的做法，否則可能導致出現新的類別，因此我們對於類別型特徵使用了OneHotEncoder或是TragetEncoder來進行類別型特徵的編碼，以利於後續模型的分類

3.對於資料target不平衡的問題，我們可以採用SMOTE技術，來擴增少數類別的資料，對於同時包含數值型以及類別型資料的Dataset，我們使用SMOTENC的技術，提供類別特徵所在的欄位，來對其進行少數類別的擴張，對於只包含數值型資料的Dataset，我們採用單純的SMOTE技術，來進行少數類別的擴張
4.對於資料筆數不足的問題，我們使用了GAN(Generative Adversarial Network)技術來去幫助我們生成更多的訓練資料，透過Discriminating Network跟Generative Network的互相訓練，來幫助我們產生更多的訓練資料

另外我們也嘗試使用了以下的特徵工程:
1.Feature Selection:對於資料集我們使用了幾個Feature Selection技術來幫助我們篩選有意義的資料，像是使用Lasso、統計檢定檢定特徵跟標籤的相關性以及使用RandomForest跟XGBoost模型來進行特徵的篩選
2.創建交互特徵:在兩個特徵間可能存在隱藏的特徵關係，我們嘗試使用數值型特徵的加減乘除來創建新的特徵，來查看模型的表現
雖然這兩個特徵工程使用的好應該是能加快模型的訓練速度、避免雜訊並且提供模型更多的特徵來嘗試提高模型的辨識準確率，但我們在訓練的過程中都發現比起使用這兩項特徵工程反而會讓我們的模型預測的結果變得更差，因此我們評估後決定不使用這兩項特徵工程技術。

B、模型以及超參數

在課堂上我們有學習到了不少的資料分類的方法，像是Randomforest、SVM，以及Bagging、Boosting、Ensemable Learning等等能夠讓模型有較好分類結果的方式，我們嘗試使用這些方式以及使用新的模型結合上方的特徵工程，以及使用一些上課中沒有提到的方法來進行二元分類的任務，以下是部分我們嘗試過的方法:

模型一、使用Neural Network(NN)、RandomForest以及XGBoost作為Base Model，分別對X_test預測其二元分類的機率，之後將其進行結合，並使用LogisticRegression作為最終的分類器，來進行最後的預測，整體的結構大致如下:

資料集	Original Training Dataset
特徵工程	SMOTE、StandardScalar
Base Model	NN、RandomForeset、XGBoost
驗證方式	5-fold validation
Meta Model	LogisticRegression
資料預測	使用Base Model預測機率後用Meta Model預測最終機率

在Base Model的部分我們使用了神經網路來作為我們其中的一個Base Model，透過神經元的連接以及學習，來捕捉資料集中的特徵，特別是對於非線性特徵能夠產生很好的效果。在這邊我們使用了BatchNormalization來解決神經層間的Inner Variance Shift問題，並且使用了Dropout以及L2 regularization來避免overfitting問題，另外我們也在神經層間選擇使用elu activation function，來達到讓數值能夠更快速收斂的效果，並使用sigmoid function作為最後的輸出。

在這裡我們也使用optuna自動調參數的方式來進行NN參數的調整，在optuna中，我們給定一個參數的區間值，然後透過10個trail來去找到屬於每個dataset中的最佳的參數，然後再使用這個參數來去進行模型的訓練，在這裡我們對於NN的兩個神經層、Dropout rate以及l2 regularization的程度來去進行自動調參數，目標是想要找到能夠擬和這個模型最佳的參數。

另外在Base Model中我們也加入了RandomForest以及XGBoost，並將NN與這兩個模型分別對X_test進行分類的預測，最後將其機率結合，使用LogisticRegression來進行最後輸出的預測。在RandomForest以及XGBoost中我們先使用了固定參數的方式來觀察其對模型的表現結果。

在特徵工程的部分，我們在這裡選擇使用最基礎的SMOTE以及Standard Scalar來對資料進行前處理。另外在這邊我們也使用了5-fold validation來對模型進行驗證，檢驗模型對於各種不同的訓練以及測試資料的預測能力。

模型二、
透過多個模型作為Base Model，分別對資料預測其機率，將其預測的結果結合起來，最後使用MLP進行最後的預測，整體的結構大致如下:

資料集	Original Training Dataset
特徵工程	SMOTENC、SMOTE、MinMaxScalar、TargetEncoder
Base Model	RandomForest、ExtraTreesClassifier、SVM、NuSVC、MLP、XGBoost、LightGBM、CatBoost、GradientBoostingClassifier、AdaBoost、HistGradientBoostingClassifier、LogisticRegression、BernoulliNB、GaussianNB、LinearDiscriminatAnalysis
驗證方式	5-fold Validation
Meta Model	LogisticRegression
資料預測	5-fold 加權平均

在這裡我們使用了多個新模型，其特色如下:

模型	特色
ExtraTreesClassifier	RandomForest的變體，建立多個極端樹進行Bagging的來達到更好的效果，以更加隨機的特徵以及樹的分裂來加快訓練速度以及降低過擬和風險
NuSVC	SVM的變體，以二次優化的方式來建構能夠最大化類間間隔的超平面，透過nu參數來控制支持向量比例的上界以及下界
MLP	最簡單的神經網路，透過Backpropagation來進行神經元中權重的更新，適合處理非線性的問題
GradientBoostingClassifier	透過多個樹的加權集成，並使用Gradient Boosting的方式來最小化loss function，以完成分類任務
AdaBoost	透過多個深度為1的tree進行加權集成形成一個強學習器，在過程中會增加錯分樣本的權重，減少正確分類樣本的權重，來關注較難分類的樣本
HistGradientBoostingClassifier	將特徵值進行分箱來降低計算成本，並使用Gradient Boosting來提升模型的效能
BernoulliNB、GaussianNB、LinearDiscraminateAnalysis	BernoulliNB根據貝氏定理來進行分類，GaussianNB假設特徵符合高斯分布來計算條件機率並分類，LinearDiscriminateAnalysis依照最大化類間差異以及最小化類內差異的方式進行分類

其中對於base model中的RandomForest、ExtratreesClassifier、SVM、NuSVC以及MLP在這裡我們使用了Bagging的方式來進行近一步的集成，其中雖然RandomFroest以及ExtratreesClassifire已經有進行內部的集成，但是我們在實作的過程中發現透過進一步的使用BaggingClassifier我們可以進一步的提升模型的效果，SVM、NuSVC以及MLP我們也嘗試利用Bagging方式來提升其效果。另外對於LogisticRegression、BernoulliNB、GaussianNB、LinearDiscraminateAnalysis這幾種分類方式我們也透過Stacking的方式來進行集成，並使用LogisticRegression來作為Stacking Model的分類器，使用這個Stacking Model來作為我們其中的一個Base Model，試著提升模型的準確率。最後我們使用Base Model對資料進行機率的預測，之後後我們將其彙整成一個輸入，並使用LogisticRegression來進行預測。

在這裡也進行了5-fold validation，來對模型進行進一步的驗證，並使用每個fold的validation auc score來作為模型預測機率的權重，透過5-fold加權平均的方式來產

出最後的預測。另外在這邊我們沒有使用超參數的調整，而是直接使用固定的參數來進行預測，因為在使用GridSearchCV以及嘗試使用optuna來進行參數調整時，我們發現其實參數其實好像對於模型的影響好像不太大，因此這裡就先使用固定的參數來嘗試，並觀察模型的表現。

模型三、
針對模型二中的結構我們進行了近一步的調整以及改善，我們加入了更多的技術、並與原本的技術結合。我們從訓練資料中加入了透過GAN生成的資料，並且簡化Base Model，以及最後改成使用Stacking方式來進行預測，其結構大致如下:

資料集	Original + Genreative(By GAN) Dataset
特徵工程	SMOTENC、SMOTE、MinMaxScalar、TargetEncoder
Base Model	RandomForest、SVM、MLP、XGBoost、LightBGM、CatBoost
驗證方式	train_test_split 0.2 for validation
Stacking Model	LogisticRegression
資料預測	用Stacking Model進行預測

在這邊我們使用了Gan來試著增強少數類別的訓練數據，由於原始數據集存在類別不平衡問題（例如，多數類別樣本顯著多於少數類別樣本），傳統機器學習模型在這種數據分佈下容易偏向多數類別，導致分類性能下降。為了解決這一問題，我們利用 GAN 生成了類似於少數類別的合成數據，從而平衡類別分佈並提升模型的泛化能力。

GAN 的實現細節:
結構：
生成器（Generator）: 輸入為隨機噪聲向量（隨機正態分佈）。通過兩層全連接層（hidden layers）轉換為與原始數據具有相同維度的輸出，模擬少數類別數據的特徵。
判別器（Discriminator）: 輸入為真實數據或生成數據。通過兩層全連接層進行二元分類，判斷輸入是來自真實數據還是生成器的合成數據。
損失函數：
採用了二元交叉熵損失函數（Binary Cross-Entropy Loss, BCE Loss）來訓練生成器和判別器。
判別器的目標是最大化識別真實數據的概率，同時最小化誤判生成數據的概率。而生成器的目標是最小化判別器識別其生成數據為假的概率。
訓練過程：
Epochs : 100 、 Batch size : 64

每個訓練周期，我們先更新判別器的權重，讓其學習如何正確區分真實數據與生成數據。接著，我們凍結判別器的參數，訓練生成器，讓其生成更加接近真實數據分佈的合成樣本。
最後，對於每個數據集中的少數類別，我們生成了 500 條合成樣本。
因為當初程式出了一些bug，漏掉了Datase26、31沒有成功生成，但因已經是所剩時間不多，故選擇跳過這兩個數據集。

我們同時也調整了Base Model以及最後的預測方法，這裡使用了六個BaseModel，並將六個Base Model以Stacking的方式進行集成，使用LogisticRegression的方式來作為最後集成的輸出，預測其二元分類的機率。

另外我們也並沒有使用超參數的調整，也把k-fold validation移除改為直接使用train data的80%作為訓練資料，另外20%作為驗證資料，因為使用6個模型進行Stacking會讓訓練時間變的更長，另外因為我們有發現將原始的訓練資料加上GAN生成訓練資料的資料，並使用Stacking進行預測後，我們可以讓每個Dataset的validation auc score基本上都在%95以上，且觀察過我們的y_prediction後，我們認為並沒有發生overfitting，因此在綜合考量時間以及我們有較高的準確性情況下，我們在這邊只使用了簡單的validation以及直接使用固定的超參數來對模型進行訓練。

C、使用的最佳超參數

模型一、

NN Model:以下的欄位由左到右分別代表:

資料集名稱、第一層神經元、L2_regularizatoin大小、Dropout_rate、第二層神經元

Dataset	n_units_1	l2_reg	dropout_rate	n_units_2
Dataset_1	42	0.000358	0.223010326	119
Dataset_2	55	0.002909	0.354669599	85
Dataset_3	81	4.59E-05	0.296132921	63
Dataset_4	94	0.001953	0.219416562	79
Dataset_5	62	0.001181	0.230823313	45
Dataset_6	86	1.31E-05	0.48714604	55
Dataset_7	32	0.001004	0.203580413	35
Dataset_8	93	0.003722	0.393089263	94
Dataset_9	103	0.000482	0.311958352	113
Dataset_10	59	0.000139	0.102695358	59
Dataset_11	97	2.27E-05	0.406152841	38
Dataset_12	126	0.000365	0.133641849	115
Dataset_13	97	0.00016	0.153495465	45
Dataset_14	41	4.93E-05	0.263955742	87
Dataset_15	120	0.002647	0.450650249	79
Dataset_16	122	0.00019	0.244939531	89
Dataset_17	103	0.000209	0.385389279	38
Dataset_18	76	0.000718	0.345562431	79
Dataset_19	46	0.000563	0.169274514	106
Dataset_20	99	4.97E-05	0.138792418	118
Dataset_21	114	9.14E-05	0.163701631	115
Dataset_22	95	0.008704	0.13287649	42
Dataset_23	33	1.00E-05	0.405584083	57

Dataset_24	121	0.009551	0.315063682	71
Dataset_25	84	0.009968	0.173916643	49
Dataset_26	114	0.0024	0.169118621	51
Dataset_27	77	0.001915	0.257277585	76
Dataset_28	95	0.007029	0.428444007	50
Dataset_29	87	0.000366	0.441289426	84
Dataset_30	87	0.000621	0.20864017	86
Dataset_31	124	0.004851	0.405108109	47
Dataset_32	106	0.000223	0.365573813	50
Dataset_33	32	0.001147	0.288610527	45
Dataset_34	62	0.000187	0.119944875	109
Dataset_35	98	0.009413	0.216773453	86
Dataset_36	128	4.69E-05	0.2654543	92
Dataset_37	44	1.20E-05	0.130066345	126
Dataset_38	71	0.006	0.159394725	87
Dataset_39	98	0.00056	0.100501747	33
Dataset_40	113	0.002262	0.474122265	73
Dataset_41	80	0.000485	0.278113032	128
Dataset_42	122	3.11E-05	0.320208343	95
Dataset_43	60	0.00062	0.109362918	67
Dataset_44	98	4.14E-05	0.152516809	72
Dataset_45	99	0.000774	0.181298197	46
Dataset_46	87	0.000804	0.215900512	32
Dataset_47	67	3.60E-05	0.292999773	50
Dataset_48	98	0.009768	0.310709747	84
Dataset_49	41	1.28E-05	0.306025818	115

RandomForest、XGBoost、LogisticRegression:

```
# Train Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_split, y_train_split)

# Train XGBoost
xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42, use_label_encoder=False, eval_metric='logloss')
xgb_model.fit(X_train_split, y_train_split)
```

```
meta_model = LogisticRegression(random_state=42)
```

其他未指定的參數使用原本的預設參數

模型二、

圖中由上到下分別為:Bagging RandomForest、Bagging ExtraTrees、Bagging SVM、Bagging NuSVC、Bagging MLP、XGBoost、LightGBM、CatBoost、GradientBoosting、AdaBoost、HistGradientBoosting、Stacking 的四個模型LogisticRegression BernoulliNB GaussianNB LinearDiscriminantAnalysis以及最後輸出的model的LogisticRegression的模型以及其參數，其中沒有設定的部分使用原本的預設參數(在這張圖中有刪掉一些像是random_state,n_jobs等的參數，完整參數如Model_2.ipynb)中檔案所示

```
rf = BaggingClassifier(estimator=RandomForestClassifier(n_estimators=400, max_depth=20)
                       ,n_estimators=10)
etc = BaggingClassifier(estimator=ExtraTreesClassifier(n_estimators=400, max_depth=20)
                        ,n_estimators=10)
svm = BaggingClassifier(estimator=SVC(gamma='auto', probability=True), n_estimators=10)
nusvc = BaggingClassifier(estimator=NuSVC(gamma='auto', probability=True), n_estimators=10)
mlp = BaggingClassifier(estimator=MLPClassifier(hidden_layer_sizes=(100,50,),
                        learning_rate='adaptive', early_stopping=True),n_estimators=10)
xgb = XGBClassifier(n_estimators=400, max_depth=20, reg_alpha=0.1, reg_lambda=1.0
                    ,eval_metric='auc',early_stopping_rounds=5)
lgbm = LGBMClassifier(n_estimators=400, max_depth=20, reg_alpha=0.1, reg_lambda=1.0,
                      min_child_samples=3,min_split_gain=0.01, early_stopping_round=5)
catboost = CatBoostClassifier(iterations=200, learning_rate=0.1, depth=6, verbose=0,
                              early_stopping_rounds=5)
gbc = GradientBoostingClassifier(n_estimators=400, max_depth=20)
adaboost = AdaBoostClassifier(n_estimators=400, random_state=51)
hist_gb = HistGradientBoostingClassifier(max_iter=400, max_depth=20,
                                         l2_regularization=0.9, early_stopping=True)
stacking_model = StackingClassifier(
    estimators=[('lg',LogisticRegression(max_iter=1000)),('bnb',BernoulliNB()),
                ('gnb',GaussianNB()),('lda',LinearDiscriminantAnalysis())],
    final_estimator=LogisticRegression(max_iter=1000),
)
meta_model = LogisticRegression(max_iter=1000)
```

模型三、

圖中由上而下分別為:RandomForest、SVM、MLP、XGBoost、LightGBM、CatBoost以及最後輸出預測的LogisticRegression的模型以及其參數，未設定的部分使用其預設的參數(在圖中有省略如random_state,n_jobs等參數，完整的參數如Model_3.ipynb中所示)

```
base_learners = [
    ('rf', RandomForestClassifier(n_estimators=1000, max_depth=30)),
    ('svm', SVC(gamma='auto', probability=True)),
    ('mlp', MLPClassifier(hidden_layer_sizes=(200,100,50,),
                          learning_rate='adaptive', early_stopping=True)),
    ('xgb', XGBClassifier(n_estimators=1000, max_depth=30, reg_alpha=0.1, reg_lambda=1.0)),
    ('lgbm', LGBMClassifier(n_estimators=1000, max_depth=30, reg_alpha=0.1, reg_lambda=1.0,
                            min_child_samples=3, min_split_gain=0.01)),
    ('catboost', CatBoostClassifier(iterations=300, learning_rate=0.1, depth=6))
]

meta_model = LogisticRegression(max_iter=1000)
```

D、模型表現比較

模型一、

5-fold validation auc score(Dataset1~10)	y_predict(Dataset_3) row 1~10	y_predict(Dataset_8) row 1~10
Average Vali	y_predict_	y_predict_
0.91238079	0.240982	0.066437
0.9991742	0.200141	0.201717
0.82527778	0.380263	0.899181
0.99167928	0.339099	0.647797
0.99901745	0.194468	0.228788
0.99229351	0.810952	0.936201
0.99904363	0.836896	0.94336
0.95954574	0.472447	0.207352
0.93656645	0.379339	0.329669
0.83324646	0.846937	0.155486

觀察我們模型的驗證結果，我們可以發現我們的5-fold validation的auc score在大部分的資料集中都有達到不錯的成果，只在少部分的資料集中結果沒有哪麼好。另外觀察Dataset3以及Dataset8資料集的y_predict成果，我們可以發現在Dataset3中的結果是沒有那麼好的，另外在Dataset8中我們在大部分的predict應該是都有達到還不錯的結果!

模型二、

這個模型是我們最後上傳的模型，在public leader board上以及private leaderboard上得到0.86336以及0.850839的auc score，我們的的模型在5-fold validation以及在y_predict上的表現如下:

5-fold validation auc score(Dataset1~10)	y_predict(Dataset_3) row 1~10	y_predict(Dataset_8) row 1~10
Validation AUC	y_predict_proba	y_predict_proba
0.937657166	0.296309923	0.095934522
0.99969752	0.344991531	0.162965941
0.914285714	0.600925738	0.739087244
0.995614598	0.470086185	0.659151237
0.998366926	0.2853677	0.148542966
0.995904769	0.685702996	0.75160698
0.998783053	0.524546152	0.890452904
0.945458333	0.321157831	0.151479801
0.884698413	0.408448312	0.142814506
0.847412305	0.671753046	0.403212362

我們有發現比較特別的是這個模型在訓練資料少的資料集(ex.Dataset3)的表現會特別不好，另外也有一些表現較不好的資料集，但整體上來說在預測上達到的還不錯的表現，並且應該是沒有遇到overfitting problem

模型三、

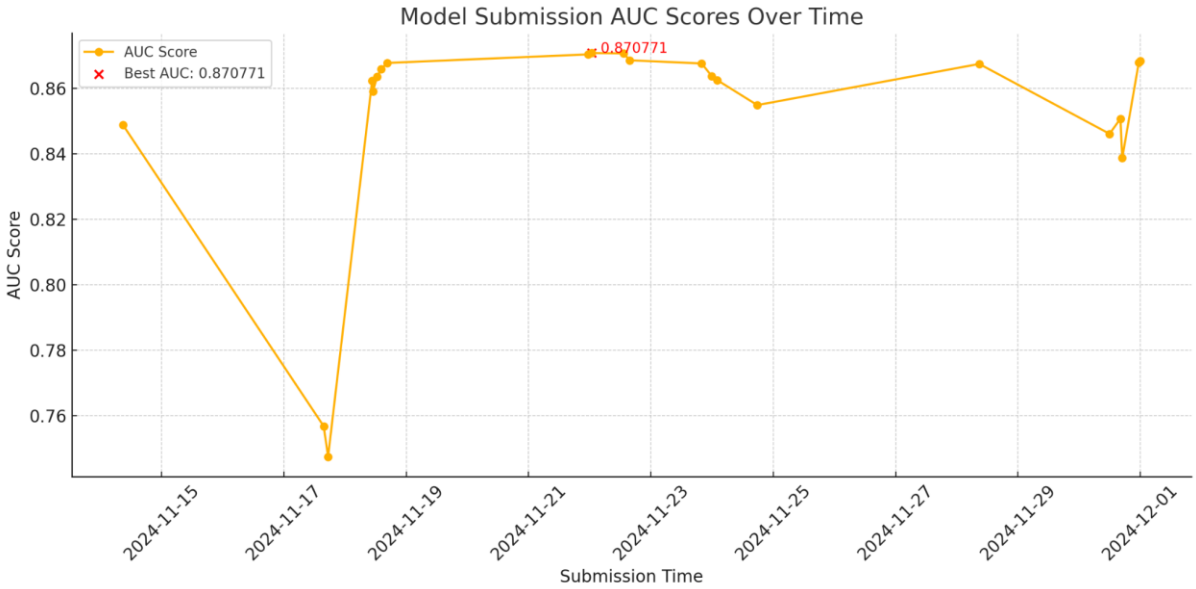
這個模型是我們最後訓練出來認為是一個表現應該會是最好的模型，但因為我們是最後一天才想到用GAN來生成訓練資料的這個方式，因此最後差了一點點沒有能夠在關閉前上傳最後一次檔案，我們的模型在validatioin以及y_predict上的表現如下:

validation auc score (Dataset1~10)	y_predict(Dataset_3) row 1~10	y_predict(Dataset_8) row 1~10
Validation AUC	y_predict_proba	y_predict_proba
0.967965256	0.877171465	0.020229463
1	0.814329639	0.030368075
1	0.975073126	0.891052702
0.988051811	0.120995877	0.906227241
0.999864517	0.002815624	0.03007241
0.998266667	0.98102206	0.949622371
1	0.952162718	0.99584177
0.999475479	0.025893132	0.019429759
1	0.192131905	0.023690794
0.982514834	0.963279982	0.050202556

整體上來說我們的模型在所有的資料集上都有很好的表現，僅有在Dataset26、Dataset31中的validation auc score沒有超過95%，但這應該是因為我們在這兩個資料集中並沒有很成功的生成由GAN所產生的訓練資料，導致並沒有達到非常好的訓練以及驗證結果，觀察上面的對於Dataset3以及Dataset8中X_test的預測結果，可以看出模型三在Dataset3中的預測結果應該是明顯優於模型二，另外在Dataset8中模型三對於每個資料的預測精準度看起來也優於模型二。此外我們的模型在訓練的過程中應該也沒有發生overfitting，如果有機會對模型的超參數進行近一步的調整或是使用k-fold validation來進行訓練集驗證的話，應該能讓模型有更好的預測結果!

E、附錄

CP1測試結果紀錄



最終排名

最終, 我們在68組參賽者中, 根據Private AUC分數最終的排名是第25名最高分為0.864254, 我們的Private AUC為0.850839